C Programming – Assignment 4: Debugging Questions Answer Key

Every buggy snippet and every corrected fix below was actually compiled and run with GCC to confirm the errors and outputs shown are real, not assumed.

D1. Identify the error(s), explain why it is wrong, and write the corrected code.

```
#include <stdio.h>
int main() {
int a, b
printf("Enter two numbers: ");
scanf("%d %d", a, b);
if (a > b)
printf("A is greater);
return 0;
}
```

Errors found (confirmed by compiling — GCC reports exactly these):

- Missing semicolon after int a, b — every statement in C must end with a semicolon.
- Missing & (address-of operator) before a and b in scanf. scanf needs the memory address of each variable to store input into; without &, it receives the variables' (garbage/uninitialized) values instead of their addresses, which is undefined behavior and can crash the program or corrupt memory.
- Unterminated string literal: "A is greater); is missing its closing double quote — a syntax error.

Corrected code:

```
#include <stdio.h>
int main() {
    int a, b;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    if (a > b)
        printf("A is greater");
    return 0;
}
```

D2. Identify the error(s), explain why it is wrong, and write the corrected code.

```
#include <stdio.h>
int main() {
int x = 10;
if (x = 5)
printf("x is 5");
else
printf("x is not 5");
return 0;
}
```

Error found:

This is a logical error, not a syntax error — the code compiles fine. = is the assignment operator, not comparison. if (x = 5) assigns 5 to x and then tests the result of that assignment (5), which is non-zero and therefore always treated as true. So the program always prints "x is 5" and silently overwrites x's original value of 10 with 5 — regardless of what x originally held. The intended comparison should use ==.

Corrected code:

```
#include <stdio.h>
int main() {
    int x = 10;
    if (x == 5)
        printf("x is 5");
    else
        printf("x is not 5");
    return 0;
}
```

With this fix, x = 10 correctly prints "x is not 5" (verified).

D3. Identify the error(s), explain why it is wrong, and write the corrected code.

```
#include <stdio.h>
int main() {
float pi = 3.14;
int radius = 5;
float area;
area = pi * radius * radius
printf("Area = %d", area);
return 0;
}
```

Errors found (confirmed by compiling):

- Missing semicolon at the end of area = pi * radius * radius — GCC reports "expected ';' before 'printf'".
- Wrong format specifier: %d expects an int, but area is a float (promoted to double when passed to printf). Using %d instead of %f reads the wrong number of bytes and reinterprets the bit pattern incorrectly, printing garbage/nonsensical output instead of the actual area.

Corrected code:

```
#include <stdio.h>
int main() {
    float pi = 3.14;
    int radius = 5;
    float area;
    area = pi * radius * radius;
    printf("Area = %f", area);
    return 0;
}
```

Verified output after fixing: Area = 78.500000

D4. Identify the error(s), explain why it is wrong, and write the corrected code.

```
#include <stdio.h>
int main() {
int num;
```

```
printf("Enter a number: ");
scanf("%d", num);
if (num % 2 = 0)
printf("Even");
else
printf("Odd");
return 0;
}
```

Errors found (confirmed by compiling):

- scanf("%d", num); is missing the & before num — GCC issues a warning here ("format '%d' expects argument of type 'int *', but argument 2 has type 'int'").
- if (num % 2 = 0) tries to assign 0 to the expression num % 2. This is a genuine compile-time error, not just a style issue: num % 2 is a computed, temporary value (not a variable), so it is not a valid target ("lvalue") for assignment. GCC reports exactly this: "lvalue required as left operand of assignment". The intended operator was the equality operator ==.

Corrected code:

```
#include <stdio.h>
int main() {
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);
    if (num % 2 == 0)
        printf("Even");
    else
        printf("Odd");
    return 0;
}
```

D5. Identify the error(s), explain why it is wrong, and write the corrected code. (Hint: look carefully at the semicolon right after the if condition.)

```
#include <stdio.h>
int main() {
int marks;
scanf("%d", &marks);
if (marks >= 90);
printf("Grade A");
else if (marks >= 75)
printf("Grade B");
else
printf("Grade C");
return 0;
}
```

Error found (confirmed by compiling — this fails to build):

The stray semicolon right after if (marks >= 90) creates an empty statement — a "do nothing" statement — as the entire body of the if. That means the if statement is already complete right there; printf("Grade A"); that follows is just an ordinary standalone statement that has nothing to do with the if, and it would run unconditionally every single time, regardless of marks.

Worse, because the if statement was already completed by the semicolon, the else that appears afterwards no longer has any if to attach to — the printf("Grade A"); statement in between is not an if-

statement, so it cannot take an else. This produces a genuine compile-time error, confirmed directly: GCC reports "error: 'else' without a previous 'if'", and the program does not compile at all.

Corrected code (remove the stray semicolon):

```
#include <stdio.h>
int main() {
    int marks;
    scanf("%d", &marks);
    if (marks >= 90)
        printf("Grade A");
    else if (marks >= 75)
        printf("Grade B");
    else
        printf("Grade C");
    return 0;
}
```

Verified: marks = 95 now correctly prints "Grade A", and marks = 80 correctly prints "Grade B".

D6. Identify the error(s), explain why it is wrong, and write the corrected code.

```
#include <stdio.h>
int main() {
char grade = 'A'
int marks = 85;
if (marks > 90)
{
grade = 'A';
else if (marks > 75) {
grade = 'B';
}
printf("Grade: %c", grade);
return 0;
}
```

Errors found (confirmed by compiling):

- Missing semicolon after char grade = 'A' — GCC reports "expected ',' or ';' before 'int'".
- The opening brace { after if (marks > 90) is never closed before the else appears. Structurally, else if (marks > 75) { ... } ends up placed inside the still-open block of the first if, as though it were an ordinary statement — but else cannot appear there, since it is not immediately preceded by a complete if statement at that point. GCC reports "expected '}' before 'else'", confirming this is a genuine brace-mismatch syntax error, not just a logical issue.

Corrected code:

```
#include <stdio.h>
int main() {
    char grade = 'A';
    int marks = 85;
    if (marks > 90) {
        grade = 'A';
    } else if (marks > 75) {
        grade = 'B';
    }
    printf("Grade: %c", grade);
    return 0;
```

```
}
```
Verified output: Grade: B (correct, since 85 > 75 but not > 90).

D7. This program is supposed to swap two numbers using a temporary variable but gives the wrong output. Find the bug.

```
#include <stdio.h>
int main() {
int a = 5, b = 10, temp;
temp = a;
a = b;
b = temp;
printf("a = %d, b = %d", a, a);
return 0;
}
```

Bug found:

The swap logic itself (temp = a; a = b; b = temp;) is completely correct — after it runs, a correctly becomes 10 and b correctly becomes 5. The bug is a simple typo in the printf call: it passes a, a instead of a, b, so both %d placeholders print the value of a. Verified output of the buggy code: "a = 10, b = 10" — the swap actually worked internally, but the output line lies about it because it never actually reads b.

Corrected code:

```
#include <stdio.h>
int main() {
    int a = 5, b = 10, temp;
    temp = a;
    a = b;
    b = temp;
    printf("a = %d, b = %d", a, b);
    return 0;
}
```

Verified corrected output: a = 10, b = 5

D8. The program should print whether a number is positive, negative, or zero, but always prints "Positive" for negative numbers too. Find and fix the logic error.

```
#include <stdio.h>
int main() {
int n = -5;
if (n >= -100)
printf("Positive");
else if (n < 0)
printf("Negative");
else
printf("Zero");
return 0;
}
```

Bug found:

The very first condition, n >= -100, is far too broad — it is true for every value from -100 upward, which includes plenty of negative numbers (like -5) as well as zero and all positive numbers. Since it is checked first in the else-if ladder and it matches almost everything, the later, more specific checks (n < 0 for

negative, and the final else for zero) are essentially never reached in practice for typical values. Verified: with n = -5, the buggy program prints "Positive", which is wrong.

Corrected code — check for positive, negative, and zero using the actual sign of n:

```
#include <stdio.h>
int main() {
    int n = -5;
    if (n > 0)
        printf("Positive");
    else if (n < 0)
        printf("Negative");
    else
        printf("Zero");
    return 0;
}
```

Verified corrected output for n = -5: Negative

D9. This program is meant to print only the name of the day matching day, but for day = 3 it prints "WednesdayThursdayInvalid" all together. Find the bug and fix it.

```
#include <stdio.h>
int main() {
int day = 3;
switch (day) {
case 1:
printf("Monday");
case 2:
printf("Tuesday");
case 3:
printf("Wednesday");
case 4:
printf("Thursday");
default:
printf("Invalid");
}
return 0;
}
```

Bug found:

None of the cases have a break statement. Once execution matches case 3, it prints "Wednesday", but with no break it then "falls through" and keeps executing the following case 4 and default blocks as well — printing "Thursday" and "Invalid" too — even though day is not 4 and the case labels below are simply ignored once execution is already inside the switch. Verified buggy output: WednesdayThursdayInvalid (exactly as described in the question).

Corrected code — add a break at the end of every case:

```
#include <stdio.h>
int main() {
    int day = 3;
    switch (day) {
        case 1: printf("Monday"); break;
        case 2: printf("Tuesday"); break;
        case 3: printf("Wednesday"); break;
        case 4: printf("Thursday"); break;
```

```
        default: printf("Invalid");
    }
    return 0;
}
```

Verified corrected output for day = 3: Wednesday (only)

D10. A student tries to use switch to compare a floating-point value and is confused why it won't even compile. Identify the exact rule being violated, and rewrite the logic correctly using if-else instead (do not simply try to "force" switch to work).

```
#include <stdio.h>
int main() {
float x = 2.5;
switch (x) {
case 2.5:
printf("Two point five");
break;
default:
printf("Other");
}
return 0;
}
```

Rule being violated (confirmed by compiling — GCC gives these exact errors):

"switch quantity not an integer" and "case label does not reduce to an integer constant". In C, the switch expression must have integer type (after the usual integer promotions), and every case label must be a constant integer expression. float is not an integer type, so neither the switch(x) itself nor the case 2.5: label is legal — this is a hard rule of the language, not something that can be worked around with a cast or a trick; switch simply cannot compare floating-point values at all.

Correct rewritten logic using if-else:

```
#include <stdio.h>
int main() {
    float x = 2.5;
    if (x == 2.5)
        printf("Two point five");
    else
        printf("Other");
    return 0;
}
```

Verified output: Two point five. Note on good practice: comparing floating-point values with == can be unreliable in general, because many decimal values (results of calculations especially) cannot be represented exactly in binary floating-point. For values entered directly as simple literals like 2.5 this exact comparison happens to work reliably, but for computed floating-point values it is safer to check whether the difference is smaller than a small tolerance, e.g. if (fabs(x - 2.5) < 0.0001), using fabs() from <math.h>.

D11. For a = 10, b = 20, c = 5, this program is supposed to print "A is largest" only when a is greater than both b and c, and print "something else" otherwise. It compiles fine but gives a logically wrong/misleading result. Trace through it, identify which if the else actually binds to, and fix it using proper braces.

```
#include <stdio.h>
int main() {
int a = 10, b = 20, c = 5;
if (a > b)
if (a > c)
printf("A is largest");
else
printf("something else");
return 0;
}
```

Bug found — dangling else — confirmed by running the exact code:

By the dangling-else rule, else always binds to the nearest preceding if that doesn't already have one — here, that is the inner if (a > c), not the outer if (a > b). So the code is actually structured as:

```
if (a > b) {
    if (a > c)
        printf("A is largest");
    else
        printf("something else");
}
```

The outer if (a > b) has no else attached to it at all. Tracing through with a = 10, b = 20, c = 5: the outer condition (10 > 20) is false, so the entire block — including the inner if/else — is skipped completely and never runs. The program produces no output whatsoever. This was verified directly by running the code: it compiled and executed with completely empty output. That is the "misleading" result — the code looks as though it handles the "otherwise" case, but that else is actually unreachable whenever a is not greater than b, so "something else" never prints for exactly the situation (a not the largest) where it was most needed.

Corrected code — combine both conditions with && and use explicit braces:

```
#include <stdio.h>
int main() {
    int a = 10, b = 20, c = 5;
    if (a > b && a > c) {
        printf("A is largest");
    } else {
        printf("something else");
    }
    return 0;
}
```

Verified corrected output for a = 10, b = 20, c = 5: something else (correct, since a is not greater than b).